Introduction to MongoDB -

Backend Development | Node.js | Express.js | MongoDB

These notes are organised from the basics to practical implementation. They are suitable for beginners learning backend development with step-by-step explanations, code examples, expected outputs, and a mini-project.

Learning objectives -

- Understand databases and why MongoDB is used.
- Create a database using MongoDB Atlas.
- Connect MongoDB with a Node.js and Express.js application.
- Perform CRUD operations: Create, Read, Update, and Delete.
- Use Mongoose schemas, models, and validation.
- Understand relationships between collections using `populate()`.
- Build a basic Student Management API.

1. Introduction to databases

A database is an organised collection of data that allows applications to store, retrieve, update, and delete information.

For example, a student management application may store student names, email addresses, courses, and marks.

1.1 Types of databases

Relational databases (SQL)

Store data in tables with rows and columns.

Types: MySQL, PostgreSQL, Oracle.

Example: A Students table with fixed columns such as id, name, and email.

NoSQL databases

Store data in flexible structures, such as documents, key-value pairs, or graphs.

Types: MongoDB, Redis, Cassandra.

Example: A student document can contain additional fields such as skills or projects.

1.2 MongoDB-

MongoDB is a NoSQL, document-oriented database that stores data in BSON documents (a binary representation of JSON-like data).

Instead of storing data in traditional tables, MongoDB organises data into databases, collections, and documents.

Student 1

```
{
  "name": "Aman",
  "age": 21,
  "course": "MERN"
}
```

Student 2

```
{
  "name": "Priya",
  "age": 22,
  "course": "Java"
}
```

Important terminology

MongoDB	**Meaning**
Database	Contains collections
Collection	Contains documents; similar to a SQL table
Document	A single record; similar to a SQL row

Field	A property within a document; similar to a column
`_id`	Unique identifier assigned to a document by default

1.3 Why use MongoDB?

- Flexible structure: Documents in the same collection can have different fields.
- Scalability: Supports scaling data storage and workloads.
- JSON-like data: Works naturally with JavaScript applications.
- Rich queries: Supports filtering, sorting, projections, and aggregation.
- Node.js integration: Commonly used in the MERN stack.

1.4 MongoDB vs MySQL

Feature	**MongoDB**	**MySQL**
Database type	NoSQL document database	Relational database
Main structure	Documents and collections	Rows and tables
Data format	BSON	Relational types
Schema	Flexible by default	Defined table structure
Relationships	References or embedded documents	Foreign keys and joins
Query language	MongoDB Query Language	SQL

Neither is universally better. The right choice depends on the application's data model and requirements.

2. MongoDB Atlas -

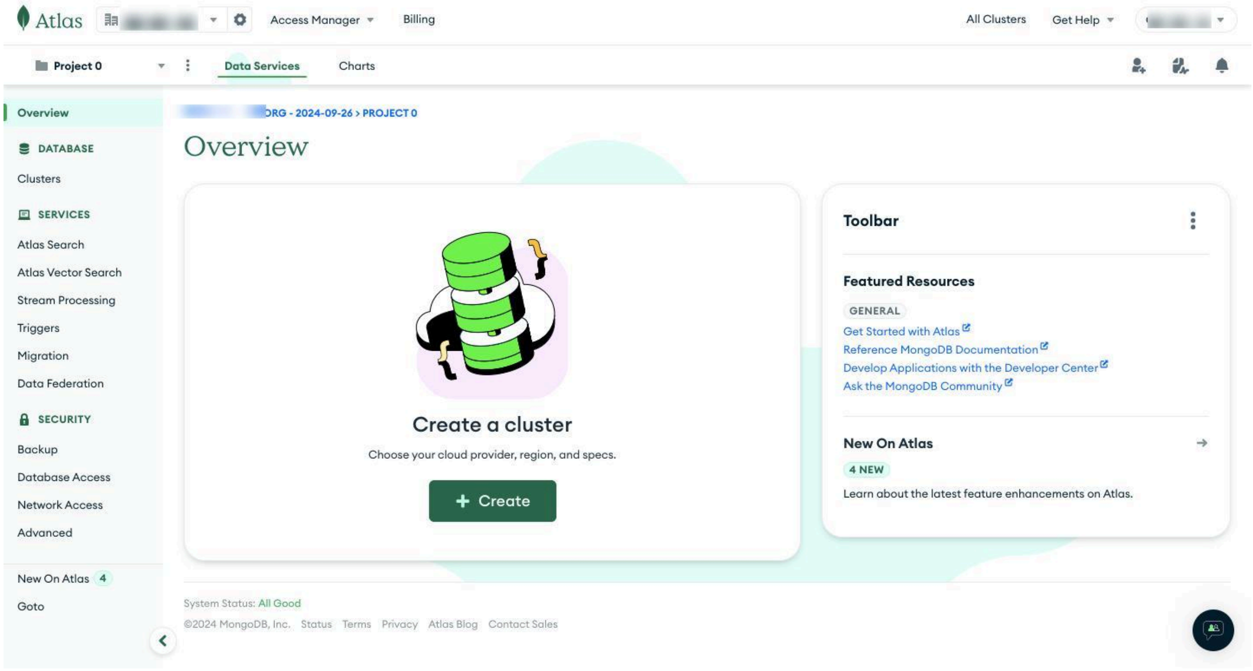
MongoDB Atlas is MongoDB's managed cloud database service. Instead of installing and maintaining the database server yourself, you can create a database cluster in the cloud and connect your application to it.

2.1 MongoDB vs MongoDB Atlas

- MongoDB: The database technology and database server.
- MongoDB Atlas: A cloud platform for hosting and managing MongoDB databases.

You can also install MongoDB locally on your computer.

2.2 Set up MongoDB Atlas step by step

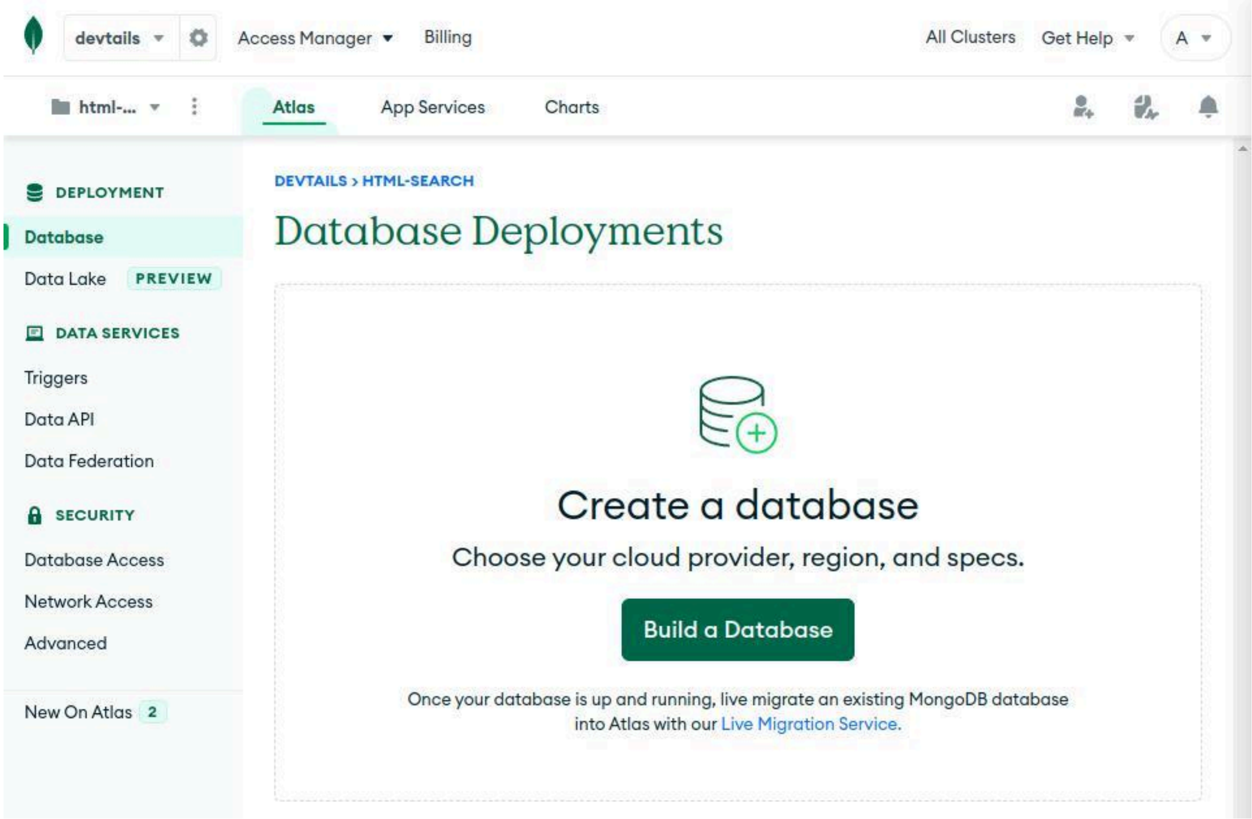

Atlas
Access Manager
Billing
All Clusters
Get Help
Project 0
Data Services
Charts
Overview
DATABASE
Clusters
SERVICES
Atlas Search
Atlas Vector Search
Stream Processing
Triggers
Migration
Data Federation
SECURITY
Backup
Database Access
Network Access
Advanced
New On Atlas 4
Goto
ORG - 2024-09-26 > PROJECT 0
Overview
Create a cluster
+ Create
Toolbar
Featured Resources
GENERAL
Get Started with Atlas
Reference MongoDB Documentation
Develop Applications with the Developer Center
Ask the MongoDB Community
New On Atlas
4 NEW
System Status: All Good
©2024 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Choose your cloud provider, region, and specs.

Learn about the latest feature enhancements on Atlas.

Step 1: Create an account

Visit the MongoDB Atlas registration page and create an account.


devtails
Access Manager
Billing
All Clusters
Get Help
A
html-...
Atlas
App Services
Charts
DEPLOYMENT
Database
Data Lake
PREVIEW
DATA SERVICES
Triggers
Data API
Data Federation
SECURITY
Database Access
Network Access
Advanced
New On Atlas 2
DEVTAILS > HTML-SEARCH
Build a Database

Database Deployments

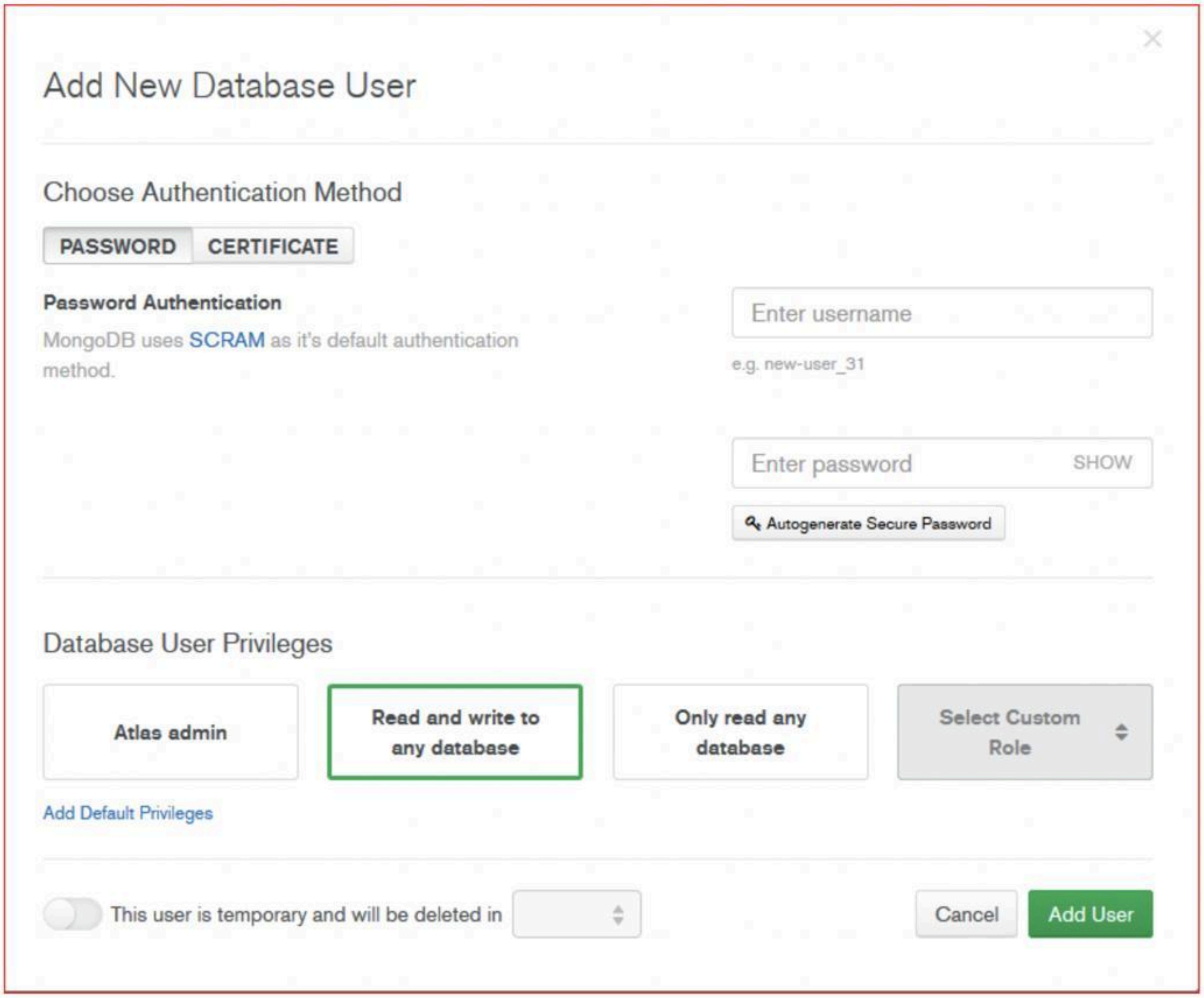
Create a database

Choose your cloud provider, region, and specs.

Once your database is up and running, live migrate an existing MongoDB database into Atlas with our Live Migration Service.

Step 2: Create a database deployment

Create a project and choose an available cluster or deployment option. Select an appropriate cloud provider and region.



Add New Database User

Choose Authentication Method

PASSWORD
CERTIFICATE

Password Authentication

MongoDB uses SCRAM as it's default authentication method.

Enter username
e.g. new-user_31
Enter password
SHOW
Autogenerate Secure Password

Database User Privileges

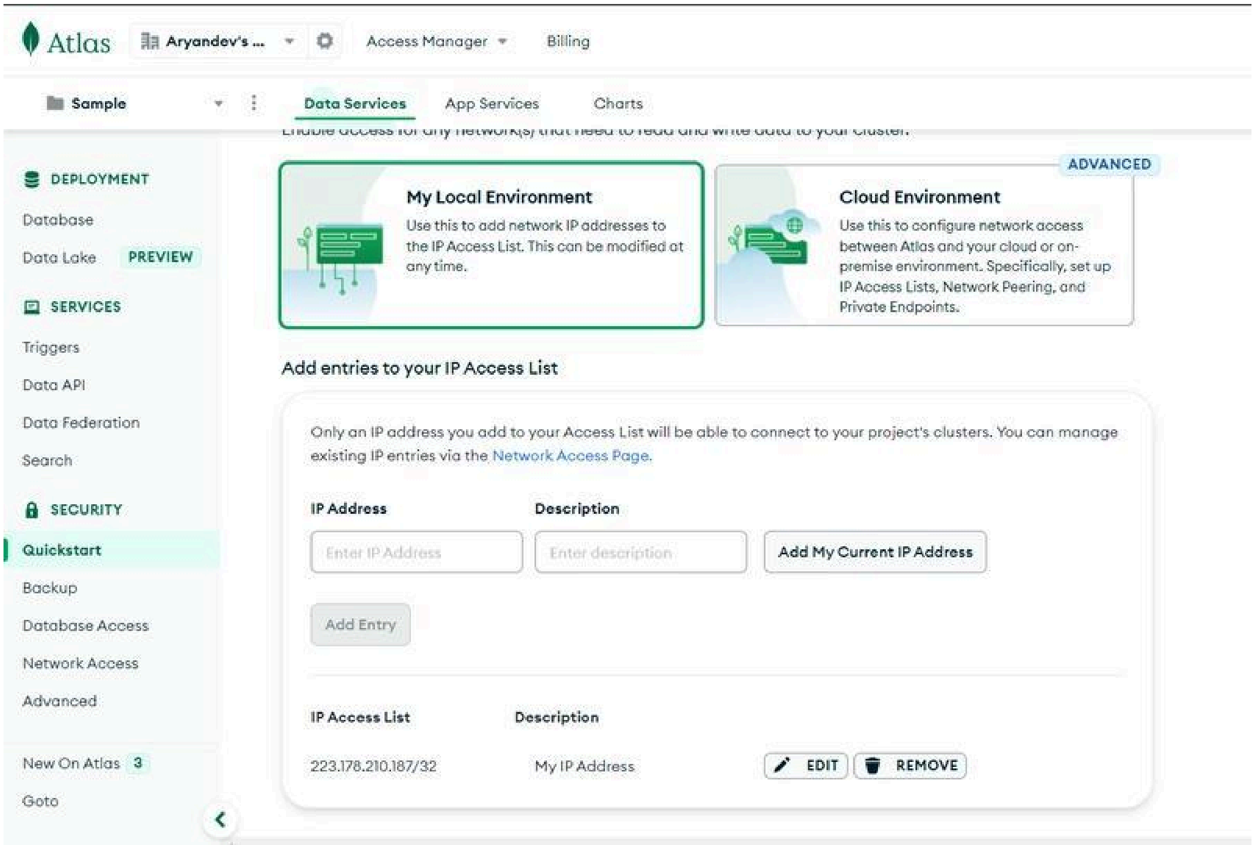
Atlas admin
Read and write to any database
Only read any database
Select Custom Role
Add Default Privileges

This user is temporary and will be deleted in

Cancel
Add User

Step 3: Create a database user

In database access settings, create a username and strong password. This is the account your application will use to authenticate.


Atlas
Aryandev's ...
Access Manager
Billing
Sample
Data Services
App Services
Charts
DEPLOYMENT
Database
Data Lake
PREVIEW
SERVICES
Triggers
Data API
Data Federation
Search
SECURITY
Quickstart
Backup
Database Access
Network Access
Advanced
New On Atlas 3
Goto
ADVANCED
Add My Current IP Address
Add Entry
EDIT
REMOVE

Enable access for any network(s) that need to read and write data to your cluster.

My Local Environment

Use this to add network IP addresses to the IP Access List. This can be modified at any time.

Cloud Environment

Use this to configure network access between Atlas and your cloud or on-premise environment. Specifically, set up IP Access Lists, Network Peering, and Private Endpoints.

Add entries to your IP Access List

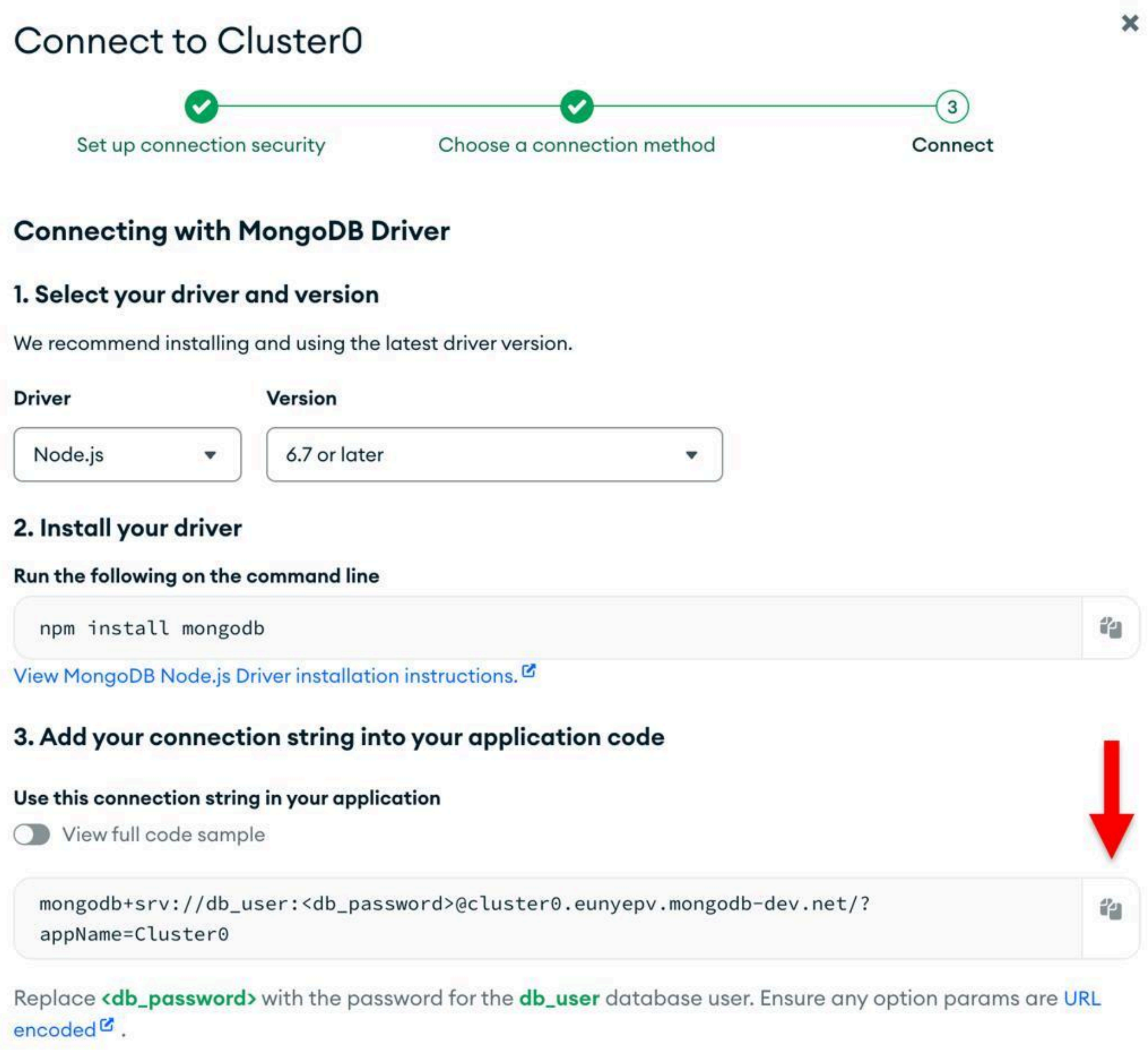
Only an IP address you add to your Access List will be able to connect to your project's clusters. You can manage existing IP entries via the Network Access Page.

IP Address: Enter IP Address
Description: Enter description

IP Access List	Description
223.178.210.187/32	My IP Address

Step 4: Configure network access

Add your current public IP address to the IP access list. Avoid allowing access from every IP address in production.



Connect to Cluster0

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver: Node.js

Version: 6.7 or later

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

View MongoDB Node.js Driver installation instructions.

3. Add your connection string into your application code

Use this connection string in your application

View full code sample

```
mongodb+srv://db_user:<db_password>@cluster0.eunyepv.mongodb-dev.net/?
appName=Cluster0
```

Replace **<db_password>** with the password for the **db_user** database user. Ensure any option params are URL encoded .

Step 5: Get the connection string

Click Connect, choose the driver connection option for Node.js, and copy the connection URI.

Official reference: MongoDB Atlas documentation.

2.3 Understanding the connection string -

An Atlas URI usually looks like this:

mongodb+srv://<username>:<password>@<cluster-host>/collegeDB?retryWrites=true&w=majority

Part	Purpose
`mongodb+srv://`	Connects using a DNS SRV record
`<username>`	Database user's username
`<password>`	Database user's password
`<cluster-host>`	Hostname of your Atlas deployment
`collegeDB`	Database name
`retryWrites=true`	Enables supported retryable writes
`w=majority`	Requests acknowledgement from a majority of voting data-bearing members

Replace the placeholders with your actual values. Do not share your real URI publicly because it may contain database credentials.

Security rule: Store the connection string in an environment variable, not directly in your source code.

3. Setting up Node.js with MongoDB

We will use the following technologies:

- Node.js: Runs JavaScript outside the browser.
- Express.js: Creates the backend server and API routes.
- MongoDB Atlas: Stores data in the cloud.
- Mongoose: Connects Node.js to MongoDB and provides schemas, models, and validation.

3.1 Create a project

First, make sure Node.js is installed.

Check the versions in your terminal:

```
node -v
npm -v
```

Create a project folder and initialise it:

```
mkdir mongodb-practice
cd mongodb-practice
npm init -y
```

Install the required packages:

```
npm install express mongoose dotenv
```

Install `nodemon` as a development dependency so the server restarts automatically when you change your code:

```
npm install --save-dev nodemon
```

3.2 Understand the packages

Package	Purpose
`express`	Creates the server and API routes
`mongoose`	Provides an interface for interacting with MongoDB
`dotenv`	Loads environment variables from a `.env` file
`nodemon`	Restarts the server when files change

3.3 Create the project structure -

Create the following files and folders:

mongodb-practice/

`server.js`

Main server

`db.js`

Database connection

`.env`

Configuration and secrets

`models/`

Mongoose models

`Student.js`

3.4 Configure environment variables

In the `.env` file, add your actual Atlas URI:

```
PORT=3000
MONGO_URI=mongodb+srv://YOUR_USERNAME:YOUR_PASSWORD@YOUR_CLUSTER/collegeDB?retryWrites=true&w=majority
```

Replace the placeholders with the details from Atlas. If your password contains reserved URL characters, encode them appropriately.

Add a `.gitignore` file:

```
node_modules/
.env
```

This prevents your environment file and installed dependencies from being committed to Git.

3.5 Connect to MongoDB using Mongoose

Create `db.js`:

```
const mongoose = require("mongoose");

async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URI);

    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
}

module.exports = connectDB;
```

Explanation-

- `require("mongoose")` imports Mongoose.
- `async` allows the function to use `await`.
- `mongoose.connect()` establishes the database connection.
- `await` waits for the connection attempt to complete.
- `try...catch` handles connection errors.

- `process.exit(1)` terminates the application if startup cannot proceed.
- `module.exports` makes the function available in another file.

Create `server.js`:

```
require("dotenv").config();

const express = require("express");
const connectDB = require("./db");

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.send("MongoDB application is running");
});

async function startServer() {
  await connectDB();

  app.listen(process.env.PORT || 3000, () => {
    console.log("Server is running");
  });
}

startServer();
```

Start the application:

```
npx nodemon server.js
```

Expected terminal output:

```
MongoDB connected successfully
Server is running
```

Open `http://localhost:3000` in your browser. You should see:

```
MongoDB application is running
```

If the connection fails, check your URI, database credentials, network access list, and internet connection.

4. Introduction to CRUD operations

CRUD stands for the four fundamental operations performed on stored data.

C — Create

Insert new documents into a collection.

Example: Register a new student.

R — Read

Retrieve existing documents.

Example: Display all students.

U — Update

Modify existing documents.

Example: Update a student's course.

D — Delete

Remove documents from a collection.

Example: Delete a student record.

MongoDB offers several ways to perform CRUD operations. We will first learn the basic MongoDB shell commands and then use Mongoose in an Express application.

5. MongoDB CRUD using MongoDB Shell

You can practise shell commands using MongoDB Compass or `mongosh`. Atlas also offers a browser-based shell option in supported deployments.

5.1 Select a database

```
use collegeDB
```

This switches to the `collegeDB` database. MongoDB may not physically create the database until data is stored in it.

5.2 Create — insert documents

Insert one student:

```
db.students.insertOne({
  name: "Aman",
  age: 21,
  course: "MERN",
  marks: 85
})
```

Insert multiple students:

```
db.students.insertMany([
  {
    name: "Priya",
    age: 22,
    course: "Java",
    marks: 90
  },
  {
    name: "Rahul",
    age: 20,
    course: "MERN",
    marks: 78
  }
])
```

MongoDB automatically generates an `_id` for each inserted document when one is not supplied.

5.3 Read — retrieve documents

Fetch all students:

```
db.students.find()
```

Fetch students whose course is MERN:

```
db.students.find({ course: "MERN" })
```

Fetch students with marks greater than 80:

```
db.students.find({ marks: { $gt: 80 } })
```

Fetch a single matching student:

```
db.students.findOne({ name: "Aman" })
```

Common query operators:

Operator	Meaning
`$gt`	Greater than
`$gte`	Greater than or equal to
`$lt`	Less than
`$lte`	Less than or equal to
`$ne`	Not equal to
`$in`	Matches any value in a specified list

For example:

```
db.students.find({
  marks: { $gte: 80 }
})
```

This retrieves students with marks of 80 or above.

5.4 Update — modify documents

Update one matching student:

```
db.students.updateOne(
  { name: "Aman" },
  { $set: { marks: 95 } }
)
```

Here:

- The first object identifies the document.
- `$set` changes the specified field without replacing the entire document.
- `updateOne()` modifies at most one matching document.

Update multiple students:

```
db.students.updateMany(
  { course: "MERN" },
  { $set: { course: "Full Stack Development" } }
)
```

5.5 Delete — remove documents

Delete one student:

```
db.students.deleteOne({ name: "Rahul" })
```

Delete all students enrolled in Java:

```
db.students.deleteMany({ course: "Java" })
```

Important: An empty filter in `deleteMany({})` deletes all documents in that collection. Always verify your filter before deleting data.

6. Introduction to Mongoose

Mongoose is an Object Data Modeling (ODM) library for Node.js and MongoDB.

It helps developers define the structure of documents, validate data, create models, execute queries, and represent relationships between documents.

6.1 Why use Mongoose?

Without Mongoose, you can use MongoDB's native Node.js driver directly. Mongoose adds useful application-level features such as:

- Schemas to define expected document structure.
- Models to interact with collections.
- Validation rules.
- Middleware and lifecycle hooks.
- Query helpers and population of references.

6.2 MongoDB vs Mongoose -

MongoDB	**Mongoose**
Database system	Node.js library
Stores the actual data	Helps your application work with the data
Uses BSON documents	Uses JavaScript objects and Mongoose documents
Has native database operations	Provides methods such as `Student.find()`
Does not require Mongoose	Requires a MongoDB connection when used with MongoDB

Remember: MongoDB stores the data. Mongoose helps your Node.js application organise and interact with it.